

HTML Tables

A Complete Reference Guide

HTML tables are a powerful way to display structured, tabular data on the web. From simple grids to complex multi-row layouts with styled cells, tables remain an essential tool in every web developer's toolkit. This guide walks through every key concept — from basic syntax to advanced styling and accessibility best practices.

Contents

1. Basic Table Structure
2. Table Tags Reference
3. Table Attributes
4. colspan and rowspan
5. Styling Tables with CSS
6. Responsive Tables
7. Accessibility Best Practices
8. Quick Reference Cheatsheet

1. Basic Table Structure

Every HTML table begins with a `<table>` tag and is built using rows (`<tr>`), header cells (`<th>`), and data cells (`<td>`). Here is the minimal structure:

```
<table>
  <tr>
    <th>Name</th>
    <th>Age</th>
    <th>City</th>
  </tr>
  <tr>
    <td>Alice</td>
    <td>28</td>
    <td>Mumbai</td>
  </tr>
  <tr>
    <td>Bob</td>
    <td>34</td>
    <td>Delhi</td>
  </tr>
</table>
```

Rendered Output:

Name	Age	City
Alice	28	Mumbai
Bob	34	Delhi

Note: `<th>` cells are bold and centered by default in most browsers, while `<td>` cells are plain left-aligned text.

2. Table Tags Reference

HTML provides several semantic tags to organize table content into logical sections. Using them properly improves both readability and accessibility.

Tag	Description
-----	-------------

<code><table></code>	Root element of the table
<code><thead></code>	Groups the header row(s); rendered at the top
<code><tbody></code>	Groups the main body rows of data
<code><tfoot></code>	Groups the footer row(s); can appear at the bottom
<code><tr></code>	Defines a table row
<code><th></code>	Defines a header cell (bold + centered by default)
<code><td></code>	Defines a standard data cell
<code><caption></code>	Provides a title or description for the table
<code><col></code>	Defines column properties (used within <code><colgroup></code>)
<code><colgroup></code>	Groups one or more columns for shared styling

```

<table>
  <caption>Employee List</caption>
  <thead>
    <tr><th>ID</th><th>Name</th><th>Role</th></tr>
  </thead>
  <tbody>
    <tr><td>1</td><td>Alice</td><td>Developer</td></tr>
    <tr><td>2</td><td>Bob</td><td>Designer</td></tr>
  </tbody>
  <tfoot>
    <tr><td colspan='3'>Total: 2 employees</td></tr>
  </tfoot>
</table>

```

3. Table Attributes

HTML table elements support several attributes to control layout and behavior. While many visual attributes are now handled by CSS, some structural ones remain important.

Attribute	Element	Description
border	<table>;	Sets border width (deprecated — use CSS instead)
cellpadding	<table>;	Space inside each cell (deprecated — use CSS)
cellspacing	<table>;	Space between cells (deprecated — use CSS)
width / height	<table>;, <td>;, <th>;	Dimensions (deprecated — use CSS)
align	<td>;, <th>;, <tr>;	Horizontal alignment (deprecated — use CSS)
colspan	<td>;, <th>;	Number of columns the cell should span
rowspan	<td>;, <th>;	Number of rows the cell should span
scope	<th>;	Declares if header applies to row, col, rowgroup, or colgroup
headers	<td>;	Associates data cell with header cells (for accessibility)
summary	<table>;	Summary of table purpose (deprecated — use <caption>;)

4. colspan and rowspan

These two attributes allow cells to span across multiple columns or rows, enabling complex grid layouts within a table.

colspan — Merging Columns

The `colspan` attribute makes a single cell occupy the space of multiple columns side by side.

```
<tr>
  <th colspan='2'>Full Name</th>
  <th>Score</th>
</tr>
<tr>
  <td>John</td>
  <td>Doe</td>
```

```
<td>95</td>
</tr>
```

Rendered Output:

Full Name		Score
John	Doe	95
Jane	Smith	88

rowspan — Merging Rows

The rowspan attribute makes a cell stretch vertically across multiple rows.

```
<tr>
  <td rowspan='2'>Math</td>
  <td>Algebra</td>
</tr>
<tr>
  <td>Geometry</td>
</tr>
```

Rendered Output:

Subject	Topic
Math	Algebra
	Geometry
Science	Physics
	Chemistry

5. Styling Tables with CSS

CSS gives you full control over table appearance. The most important property to know is `border-collapse`, which determines whether cell borders are separate or merged into a single border.

Essential CSS Properties

```
table {
  width: 100%;
  border-collapse: collapse; /* merge double borders */
  font-family: Arial, sans-serif;
}

th, td {
  padding: 10px 14px;
  border: 1px solid #ccc;
  text-align: left;
}

th {
  background-color: #2e4a7a;
  color: white;
}

tr:nth-child(even) {
  background-color: #f3f5fb; /* zebra striping */
}

tr:hover {
  background-color: #dce8f5; /* hover highlight */
}
```

Colgroup — Column-Level Styling

The `<colgroup>` element lets you apply styles to entire columns without repeating attributes on every cell.

```
<table>
  <colgroup>
    <col style='background-color:#eef3ff; width:40%'>
    <col style='background-color:#fff8e1; width:30%'>
    <col style='background-color:#e8f5e9; width:30%'>
  </colgroup>
  ...
</table>
```

6. Responsive Tables

On small screens, wide tables overflow their container. There are two common approaches to handle this gracefully:

Method 1: Horizontal Scroll Wrapper

```
<div style='overflow-x: auto; '>
  <table>
    ...
  </table>
</div>
```

Wrap the table in a scrollable div. The user can horizontally swipe to see all columns. Simple, no layout change required.

Method 2: Card/Stack Layout on Mobile

```
@media (max-width: 600px) {
  thead { display: none; }
  tr { display: block; margin-bottom: 12px; border: 1px solid #ccc; }
  td {
    display: block;
    text-align: right;
  }
  td::before {
    content: attr(data-label);
    float: left;
    font-weight: bold;
  }
}
```

On mobile, each row transforms into a card, with column names shown as labels using `data-label` attributes. This requires adding `data-label` to every `<td>`.

7. Accessibility Best Practices

Semantic markup makes tables usable for screen readers and assistive technologies. Follow these practices to ensure your tables are accessible:

#	Practice	Example
1	Use to describe the table	Monthly Sales Report
2	Use scope on elements	Name
3	Use , ,	Proper semantic grouping
4	Avoid using tables for layout	Use CSS Grid or Flexbox instead
5	Use headers + id for complex tables	Quarter
6	Avoid empty header cells	Every should have meaningful text
7	Don't rely on color alone	Add text labels or icons for context

8. Quick Reference Cheatsheet

Task	Code / Technique
Create a basic table	...
Add a header row	...
Add a caption	My Table
Merge 2 columns	...
Merge 3 rows	...
Remove double borders	<code>table { border-collapse: collapse; }</code>
Zebra striping	<code>tr:nth-child(even) { background: #f0f0f0; }</code>
Hover highlight	<code>tr:hover { background: #dce8f5; }</code>
Fixed column widths	<code>table { table-layout: fixed; }</code>
Scrollable on mobile	...
Scope for accessibility	<code>or scope="row"</code>
Style whole column	

Remember: Use tables for tabular data only — never for page layout. For layout, use CSS Flexbox or Grid instead.